

Beam Search Búsqueda de Haz

Beam Search: Una Búsqueda Heurística con Haz de Luz

Explorando el Espacio de Búsqueda con un Enfoque Selectivo



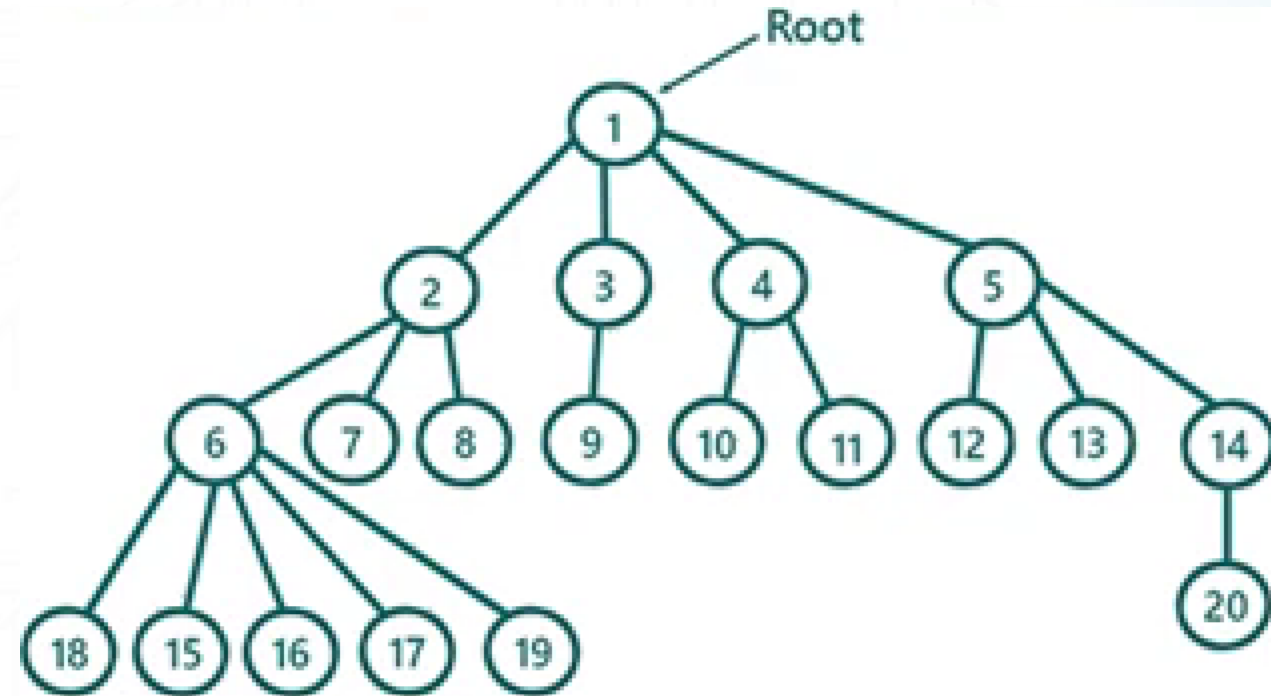
- La búsqueda de haces “Beam Search” es un algoritmo de búsqueda heurística que explora un grafo al expandir el nodo más prometedor en un conjunto limitado.
- Beam search es una optimización del algoritmo “best-first search” que reduce sus requisitos de memoria.
- En la búsqueda de haces, es un algoritmo codicioso , en el cual solo se mantienen como candidatos un número predeterminado de las mejores soluciones parciales.

Beam Search

Es un algoritmo heurístico que examina un grafo extendiendo el nodo mas prometedor en un espacio limitado.

El número de nodos “n” representa el “beam-width”

Este algoritmo solo mantiene el menor número de nodos en la lista abierta.



Componentes – Beam Search

El algoritmo “beam search” toma los componentes del árbol como entrada

1. El problema es usualmente representado mediante un grafo y contiene el conjunto de nodos en el cual pueden existir una o varios objetivos
2. **El conjunto de reglas heurísticas para la poda:** Son reglas específicas del dominio del problema y eliminar los nodos desfavorables de la memoria con respecto al dominio del problema
3. **Una memoria con una capacidad disponible limitada:** La memoria es donde se almacena el "haz", la memoria está llena y se agregará un nodo al haz, el nodo más costoso se eliminará, de modo que no se exceda el límite de memoria

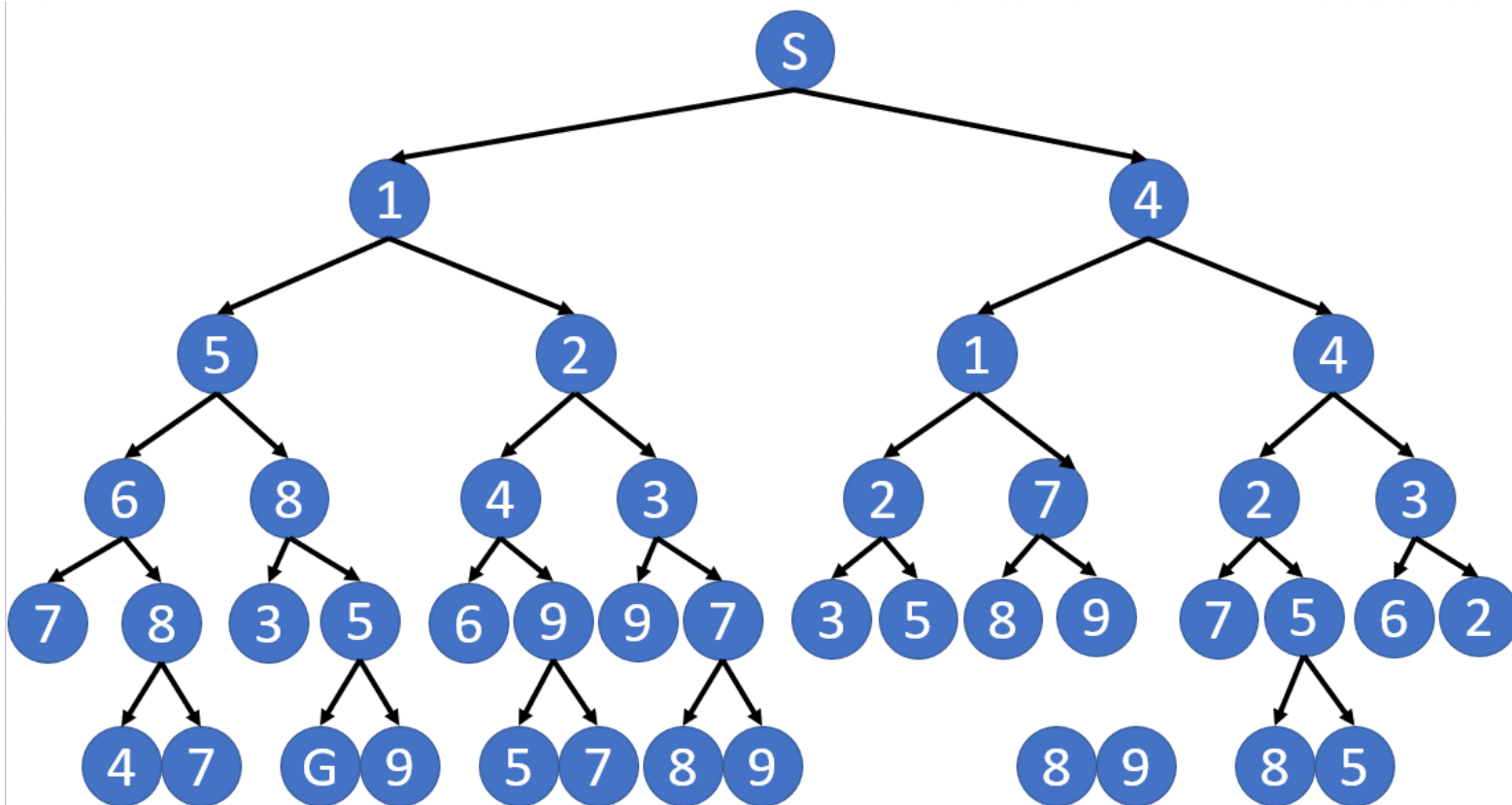
Beam Search

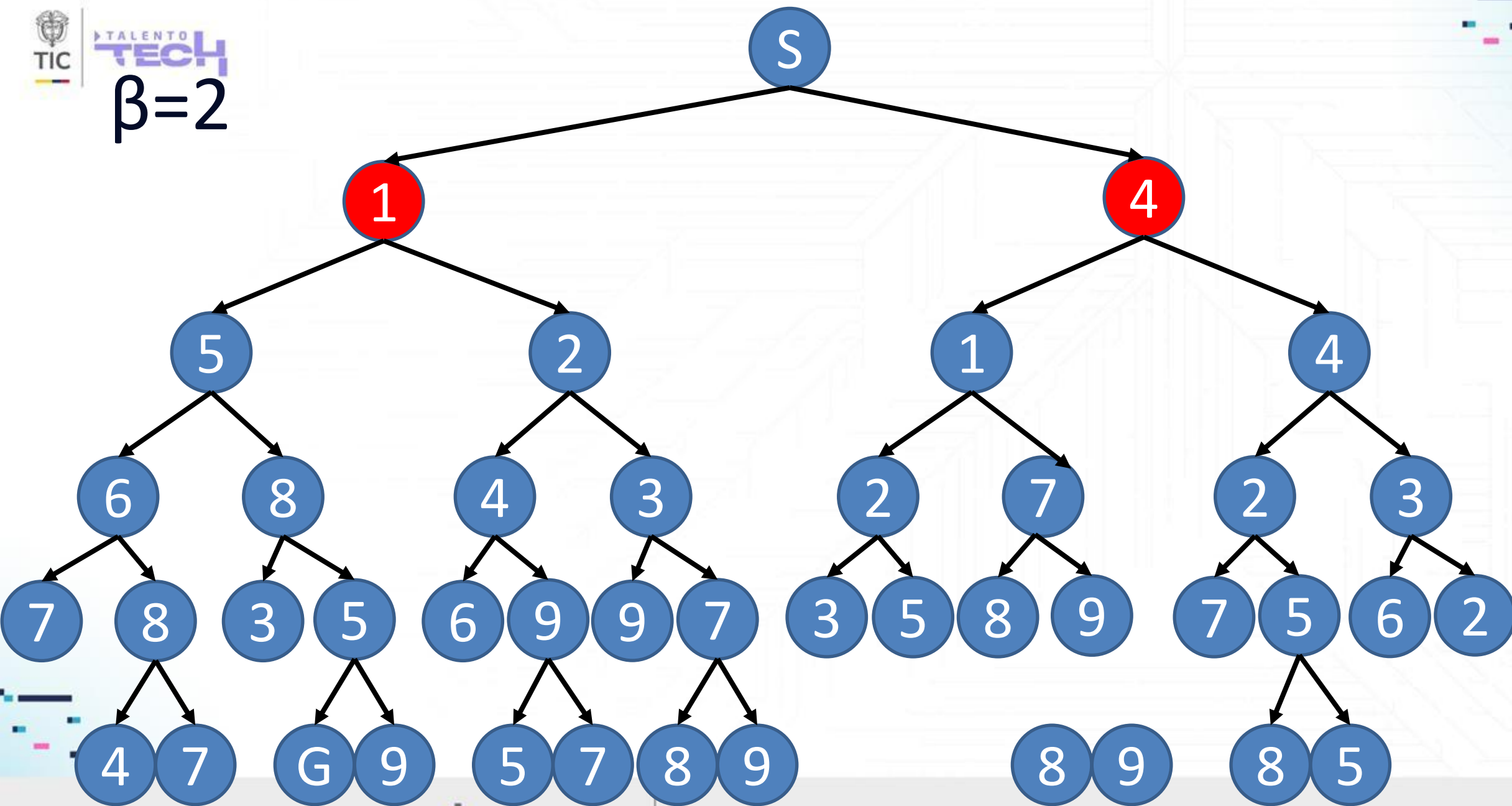
Si el “beam size” es 1 entonces este algoritmo sería “Best First Search”

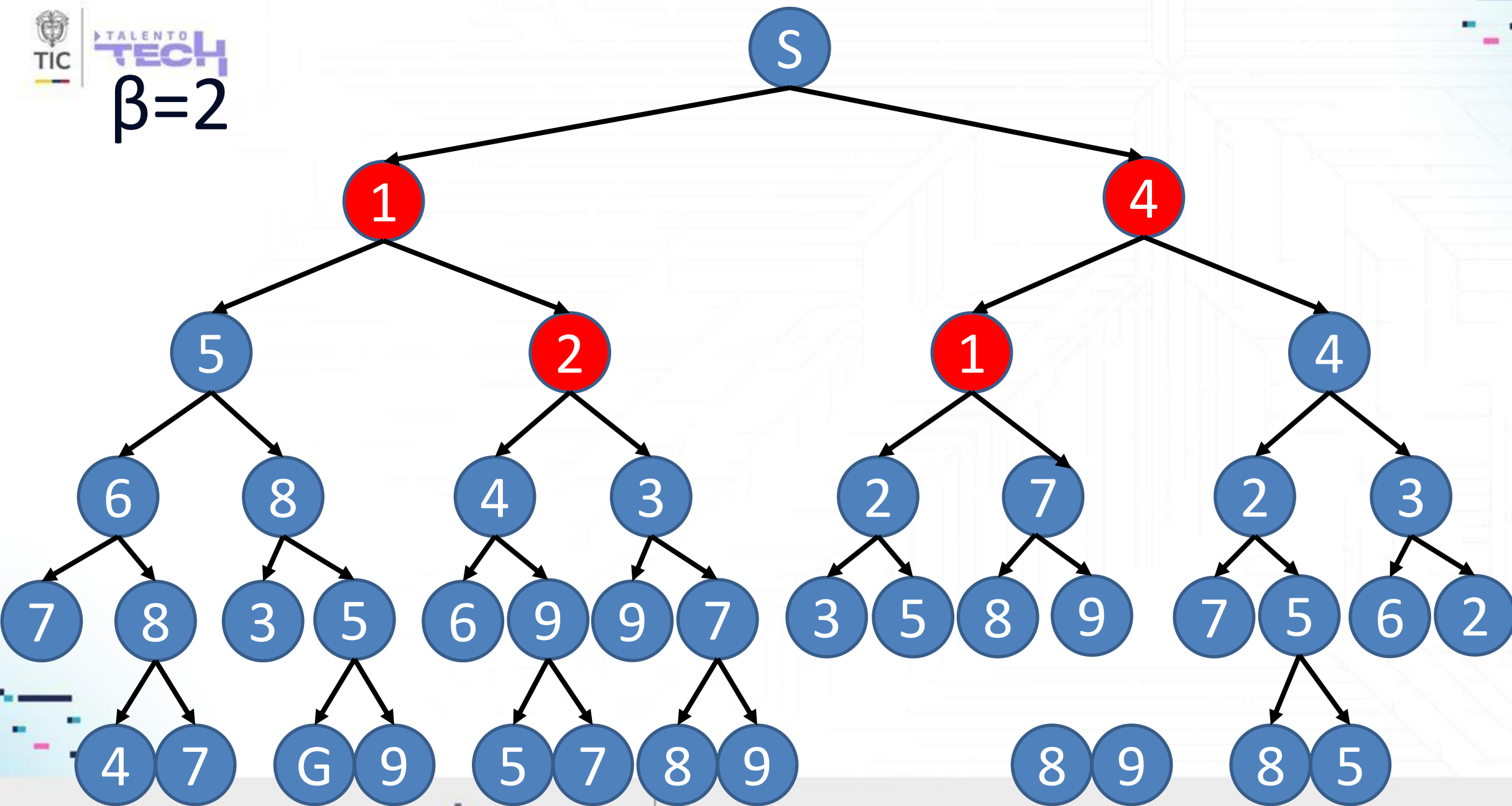
Cuando es comparado con el algoritmo de búsqueda A*, la complejidad de tiempo y espacio será mucho mas reducido.

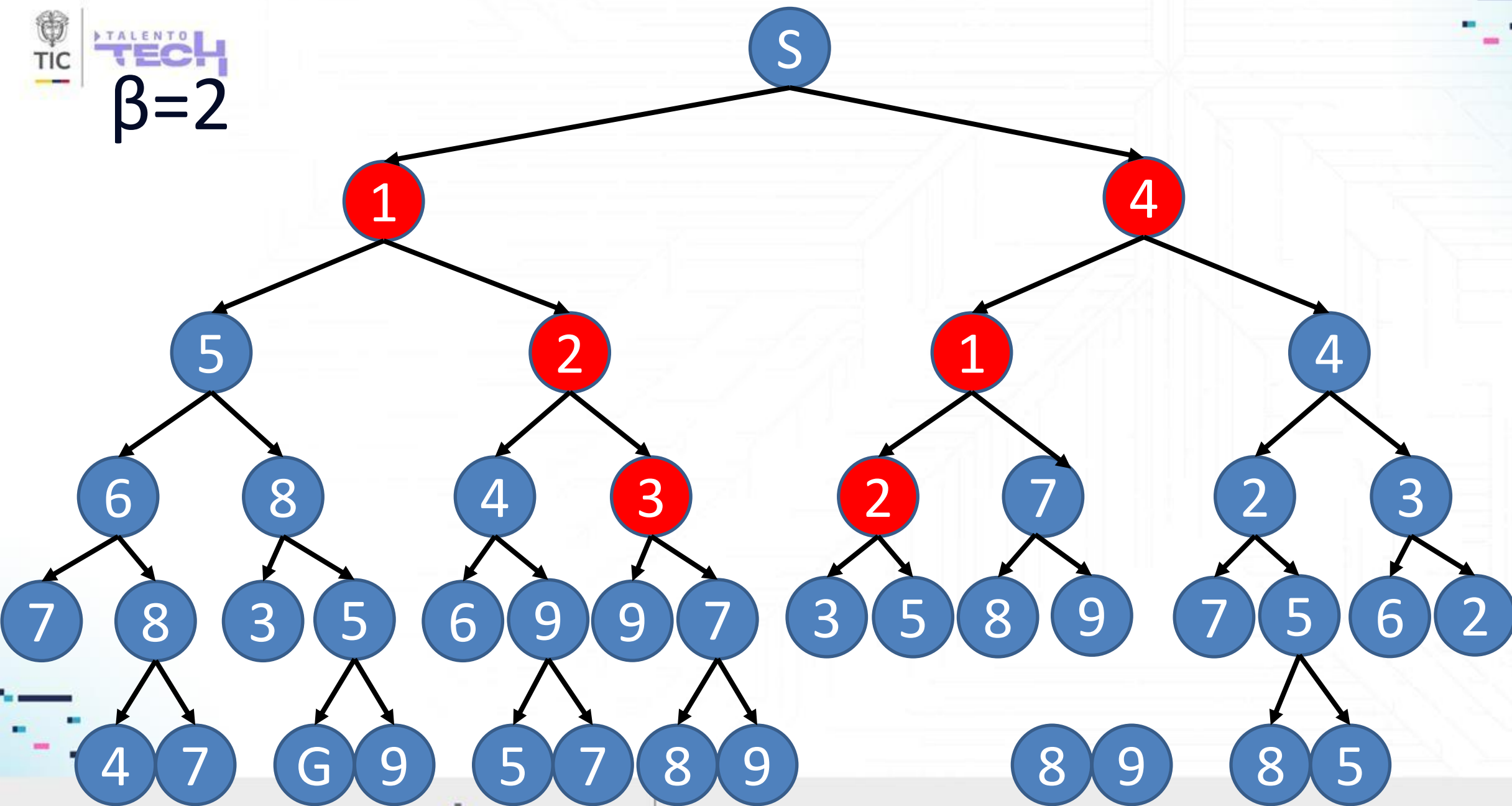
Ejemplo - Explicación

El valor de beta corresponde a la cantidad máxima de hijos que posee un nodo en el árbol

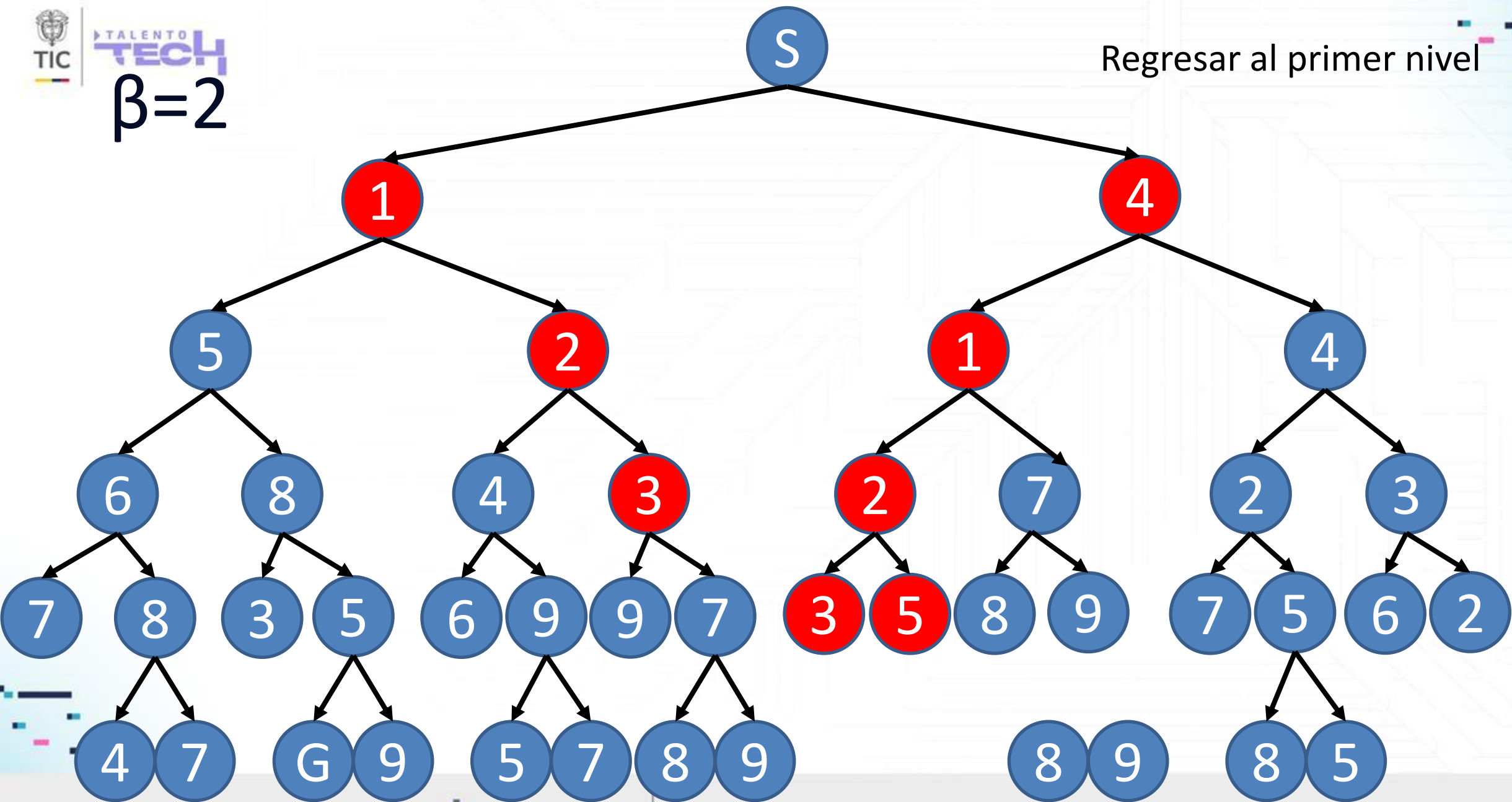






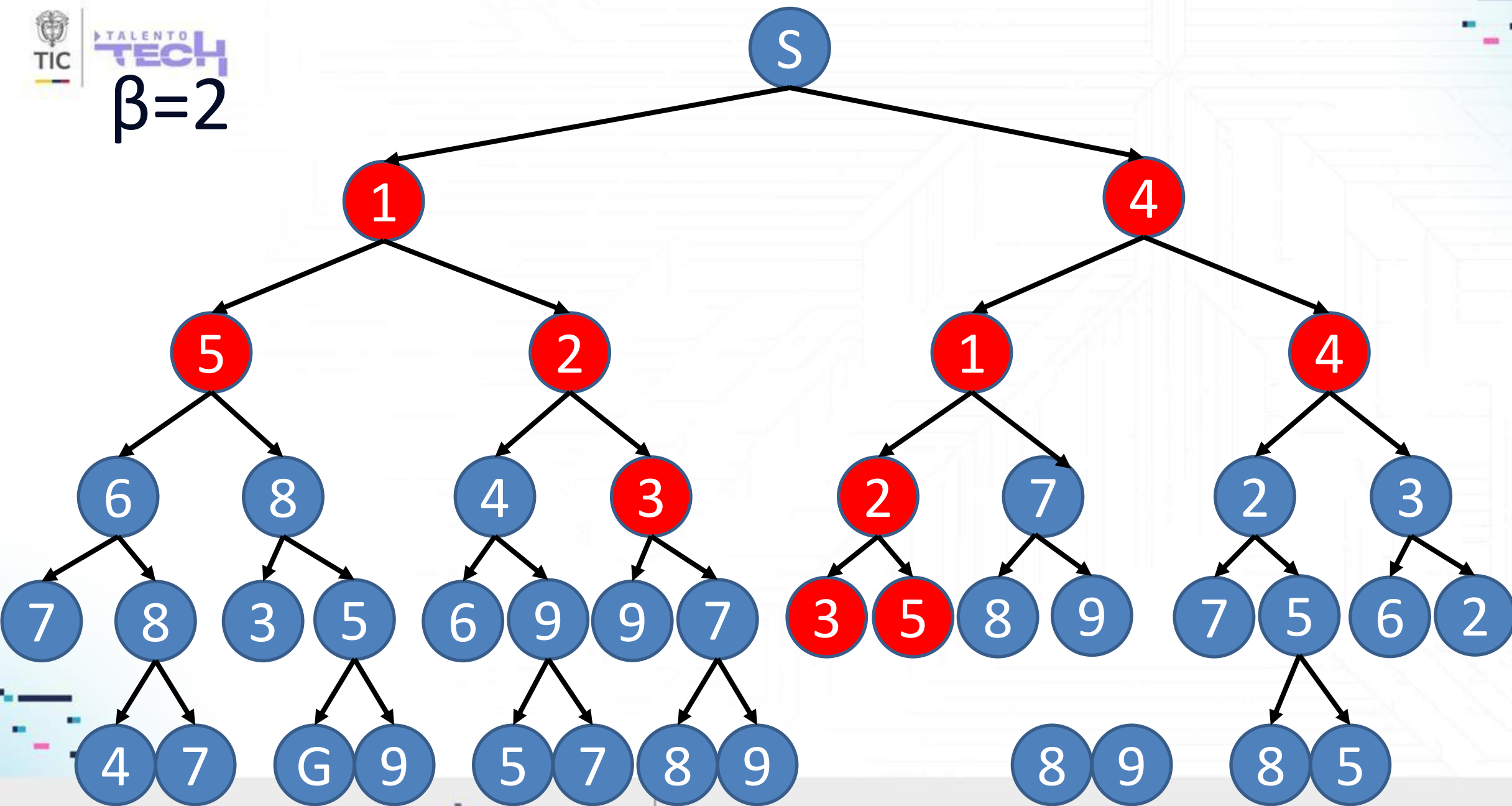


Regresar al primer nivel



Regresar al primer nivel,
ya están todos visitados

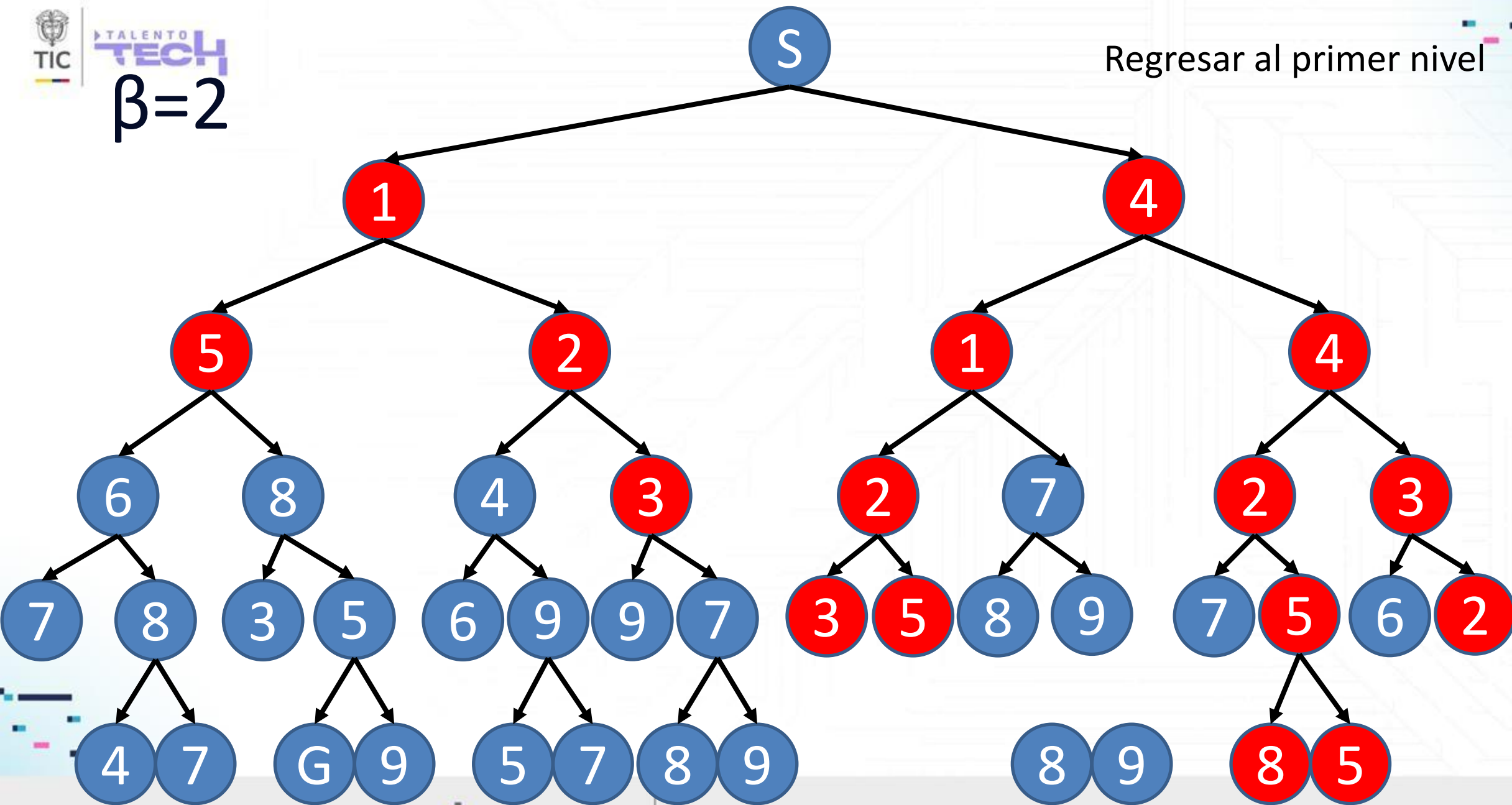


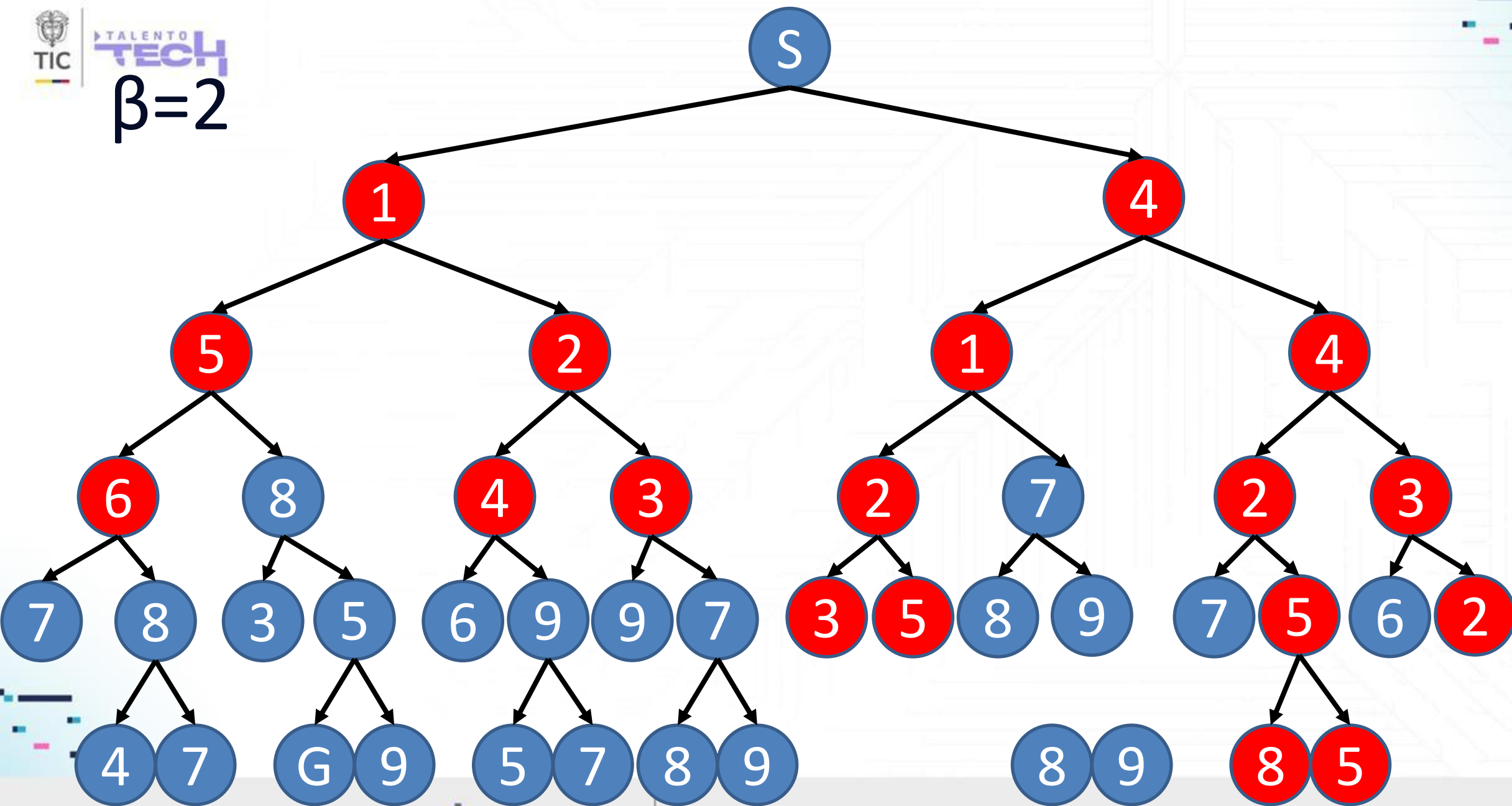




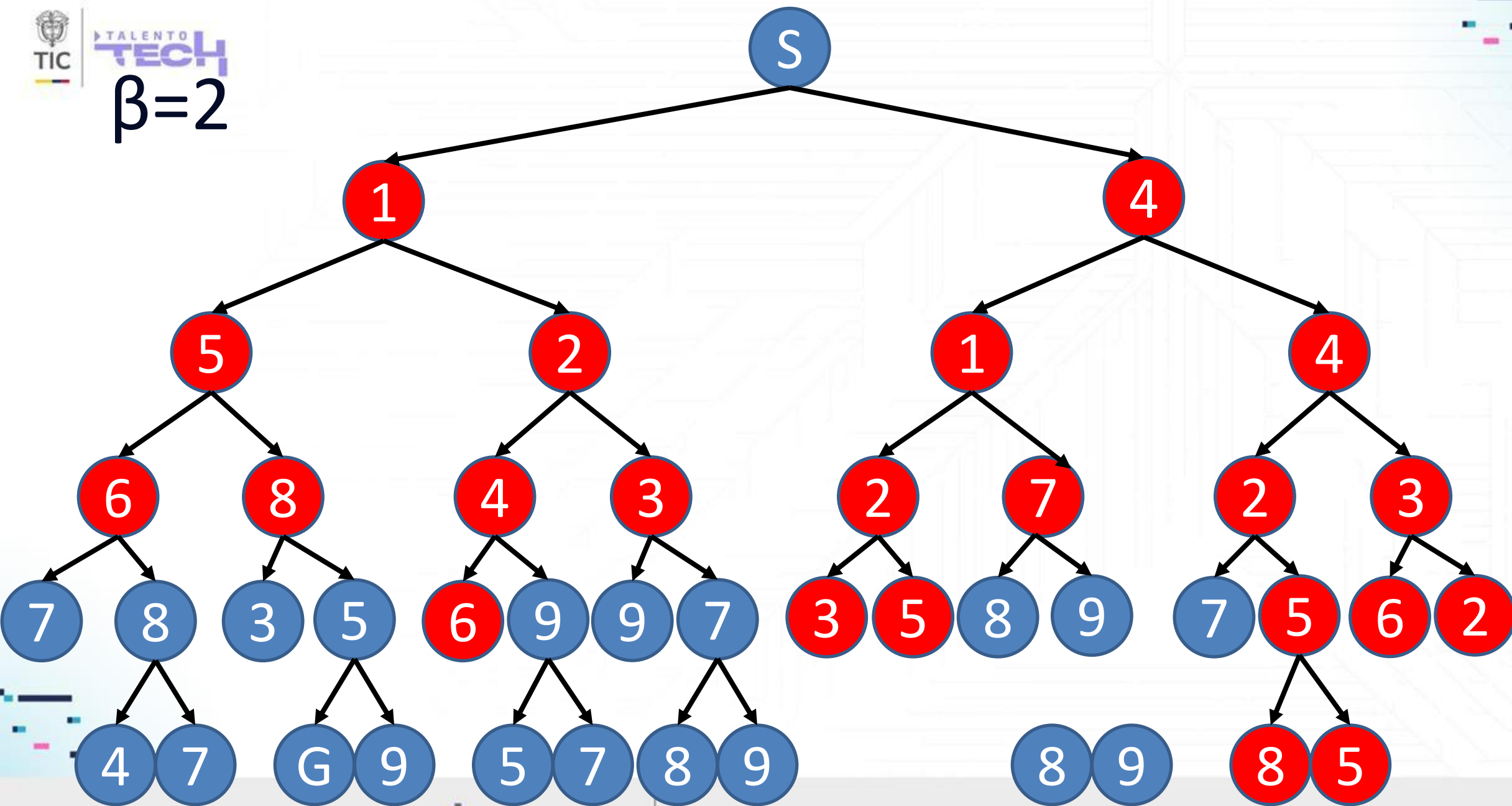


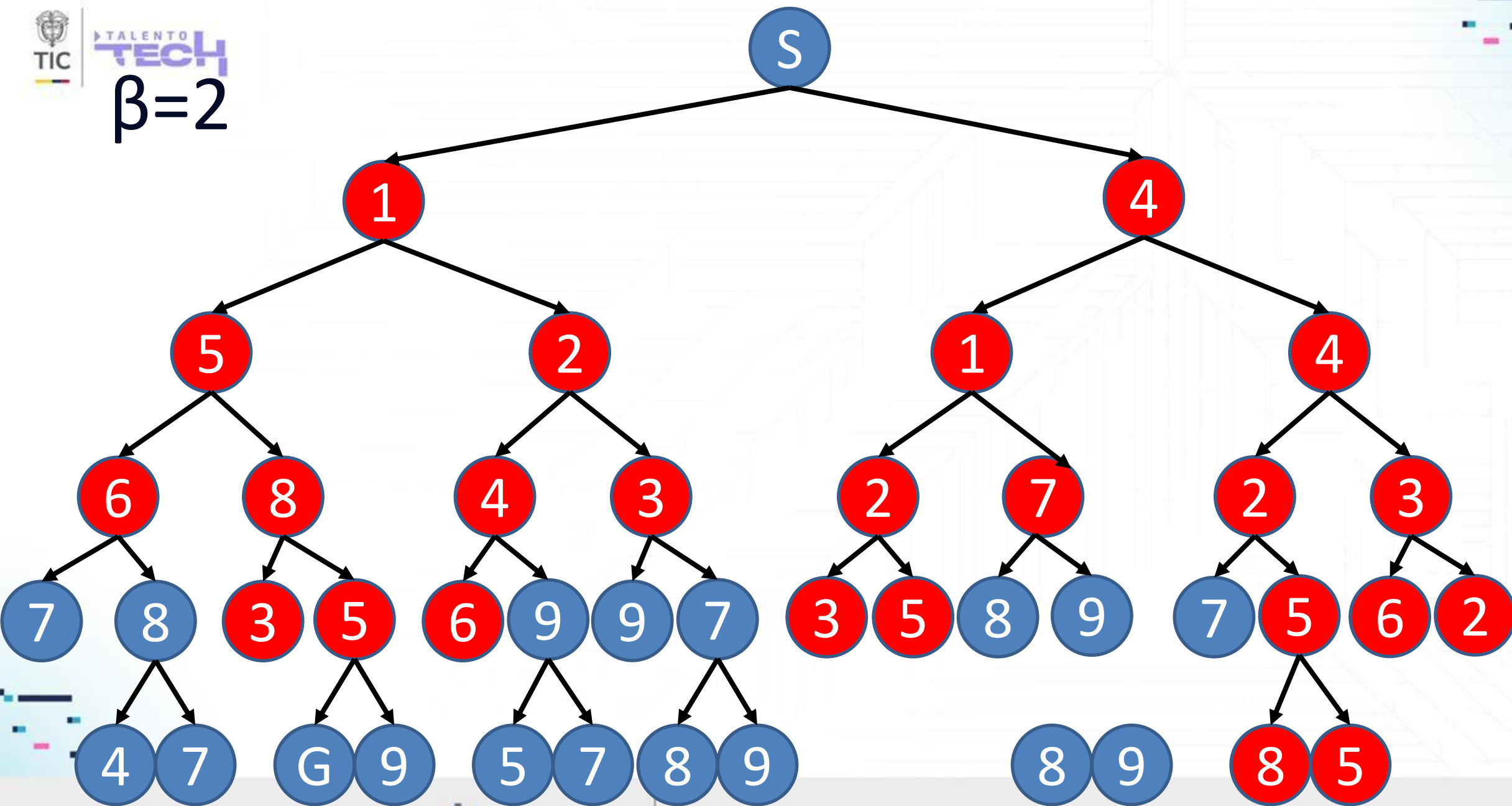
Regresar al primer nivel





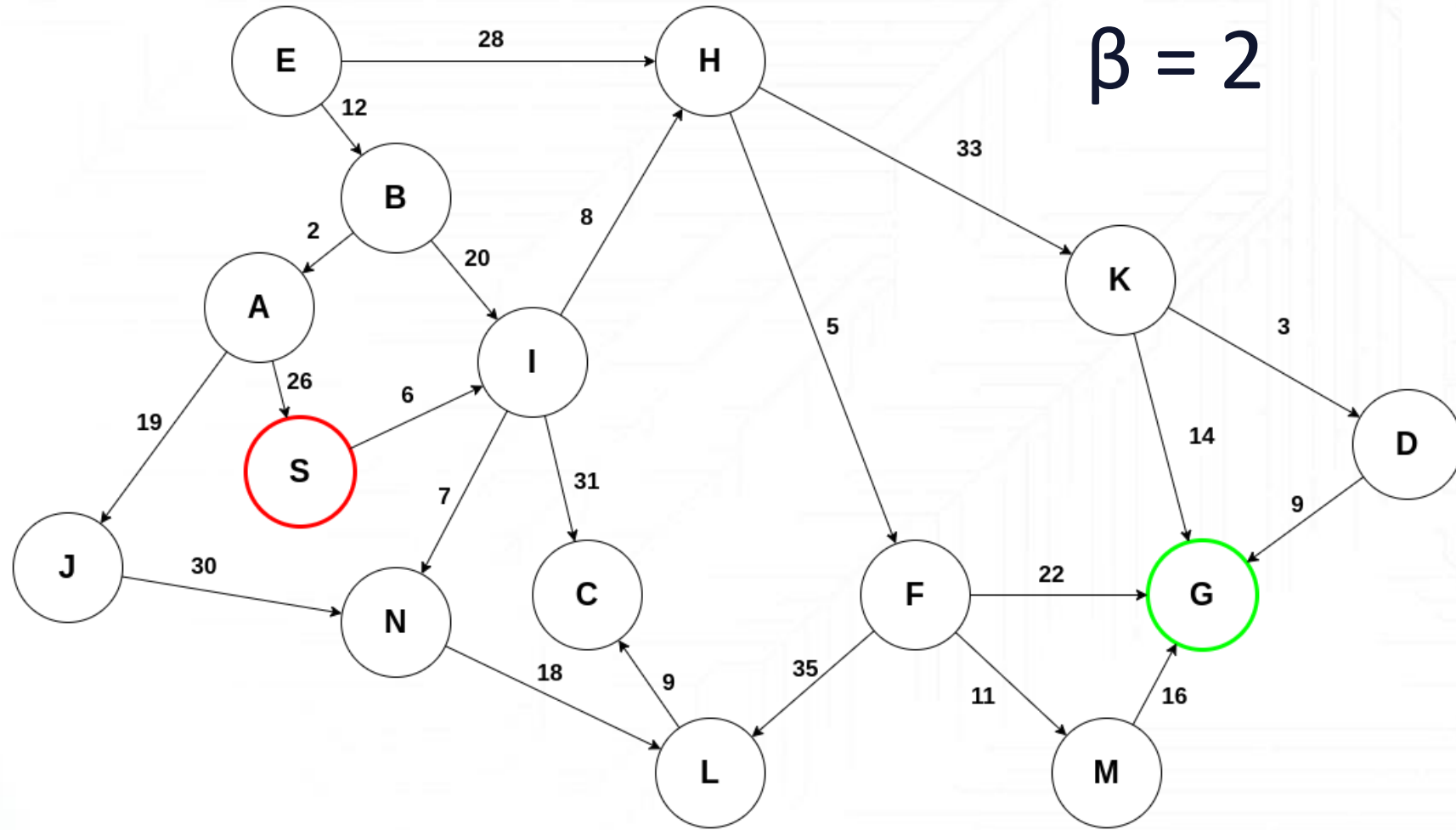






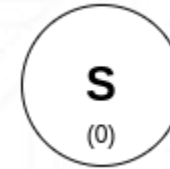
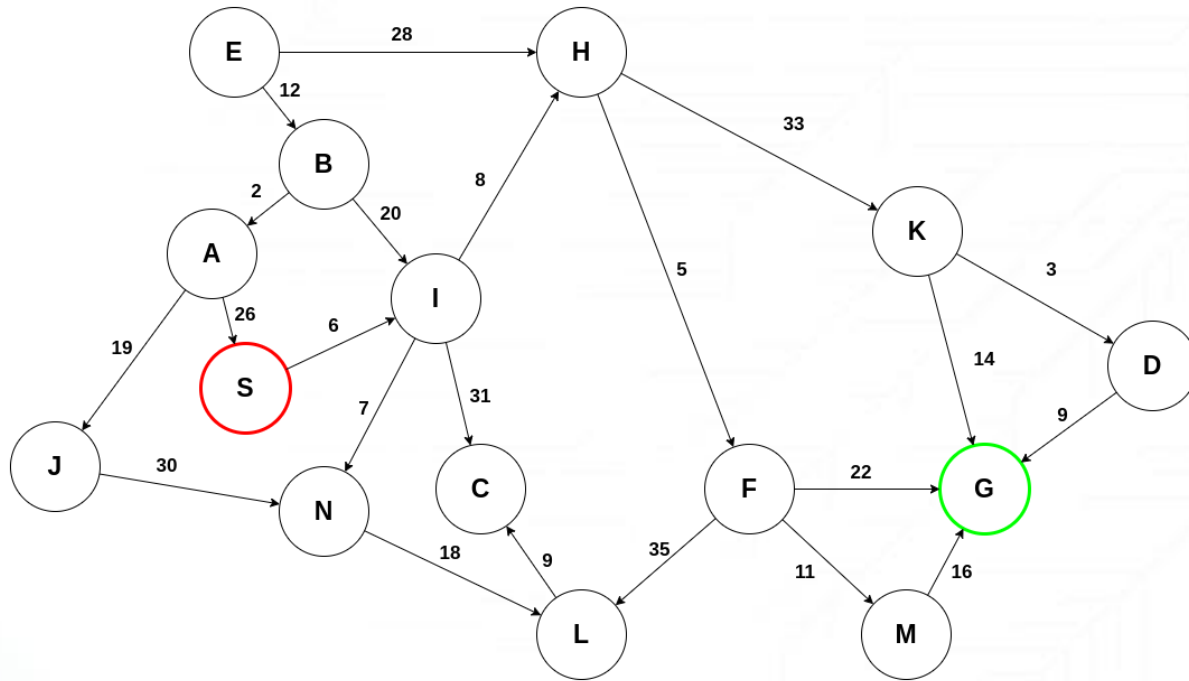


Ejemplo 2 - Explicación



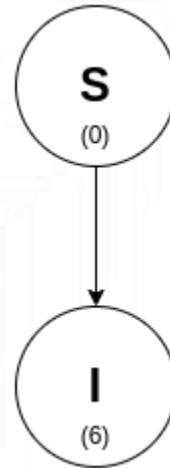
Ejemplo 2 - Explicación

$$\beta = 2$$



Node[cost]
I[6]

Closed List
S

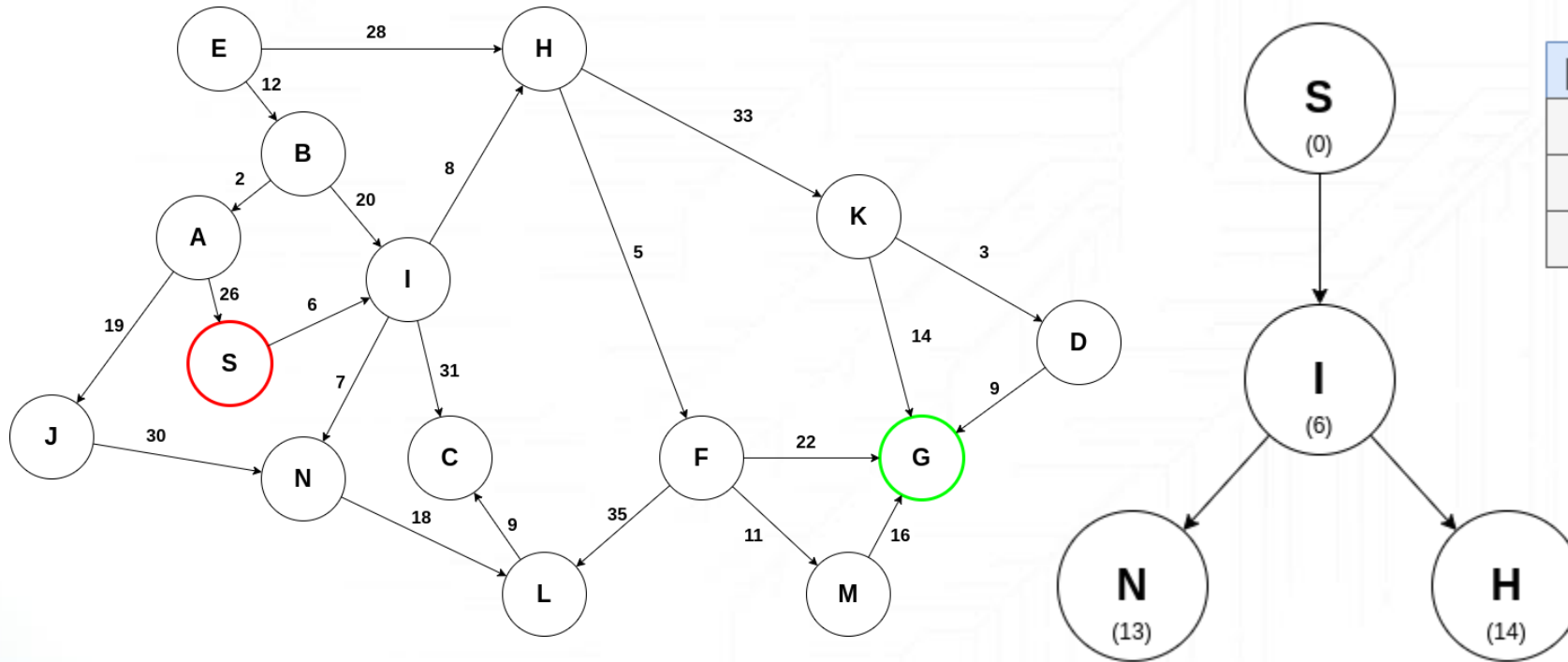


Node[cost]
N[13]
H[14]
C[37]

Closed List
S
I

Ejemplo 2 - Explicación

$$\beta = 2$$

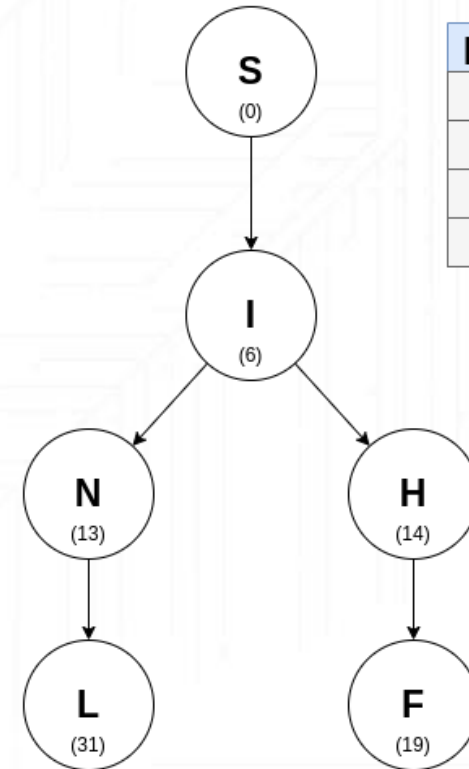
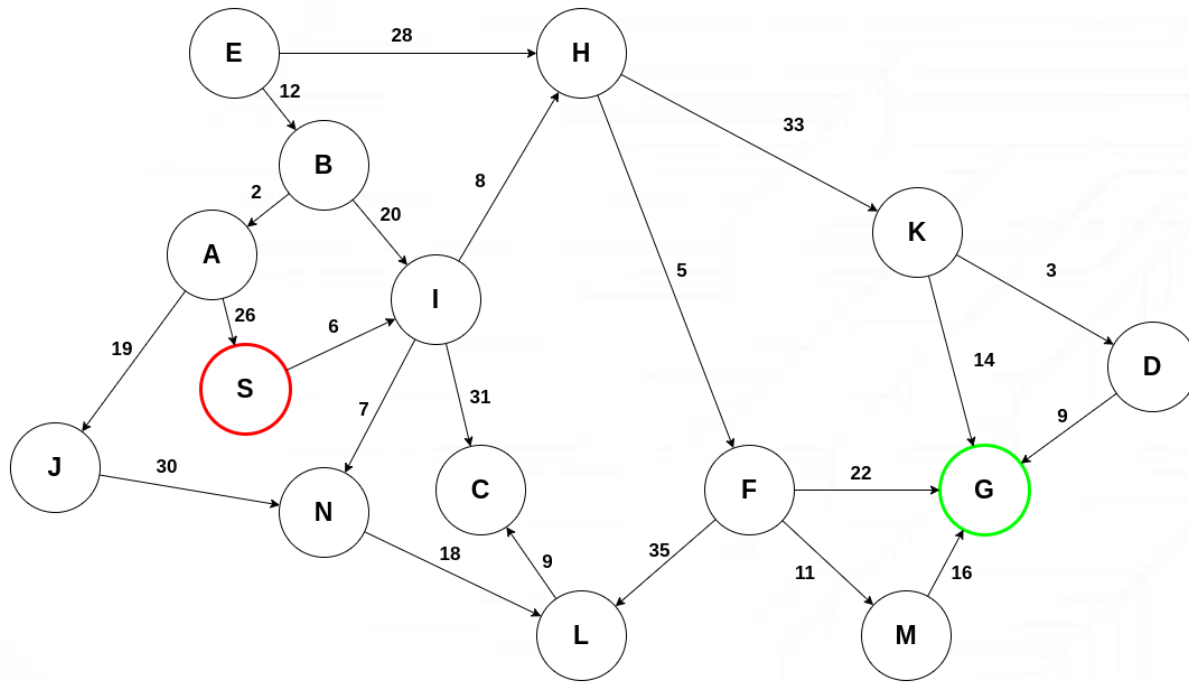


Node[cost]
F[19]
L[31]
K[47]

Closed List
S
I
N
H

Ejemplo 2 - Explicación

$$\beta = 2$$

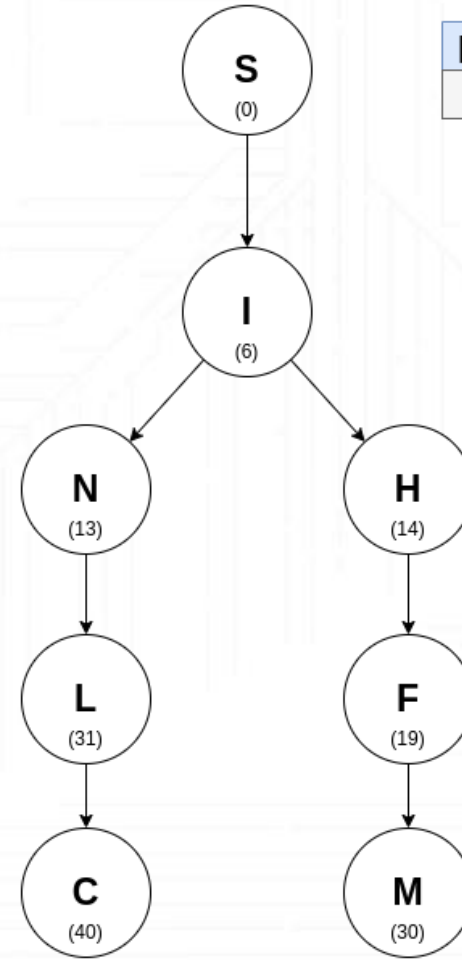
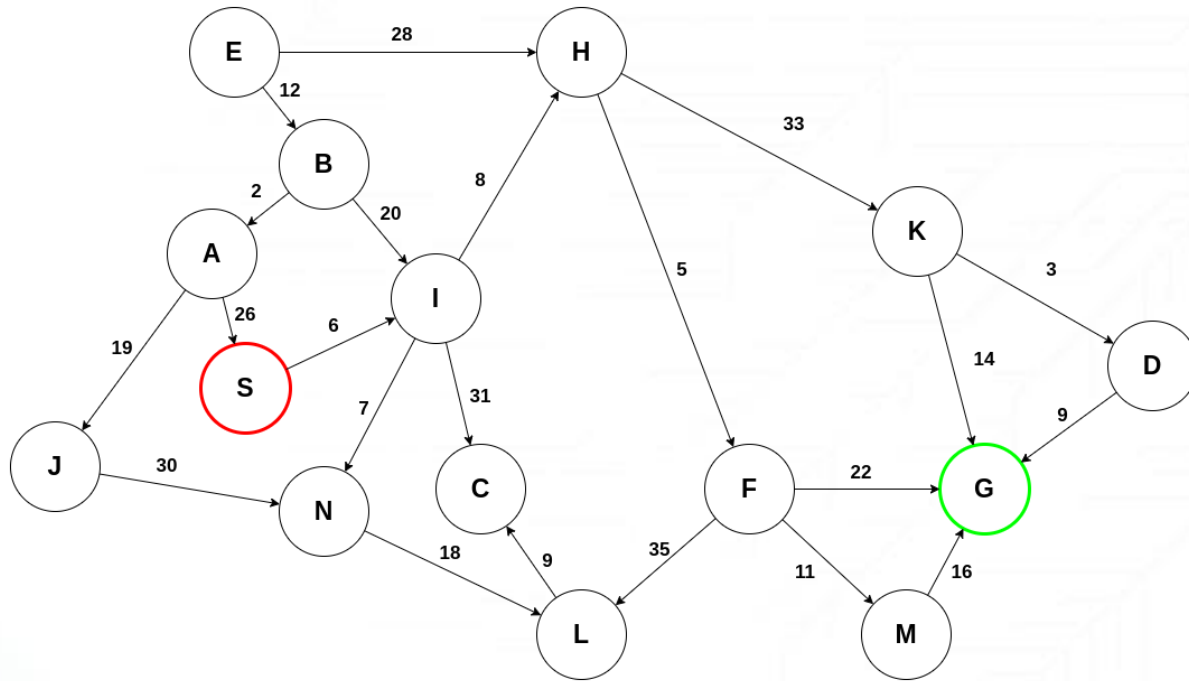


Node[cost]
M[30]
C[40]
G[41]
L[54]

Closed List
S
I
N
H
F
L

Ejemplo 2 - Explicación

$$\beta = 2$$

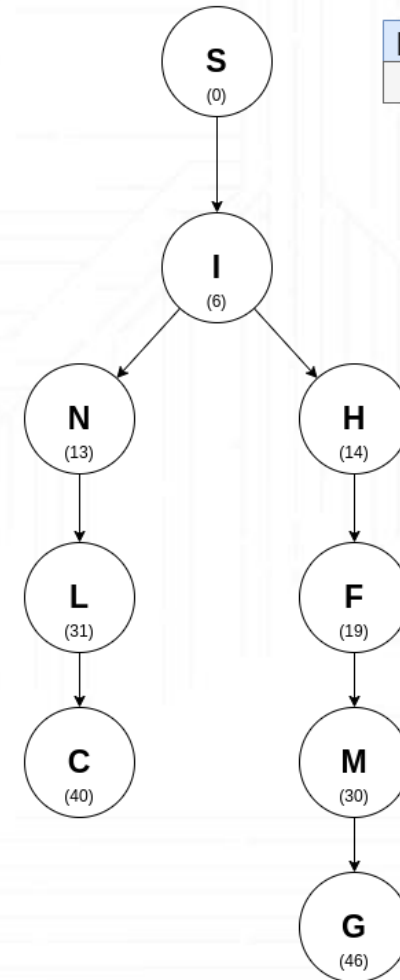
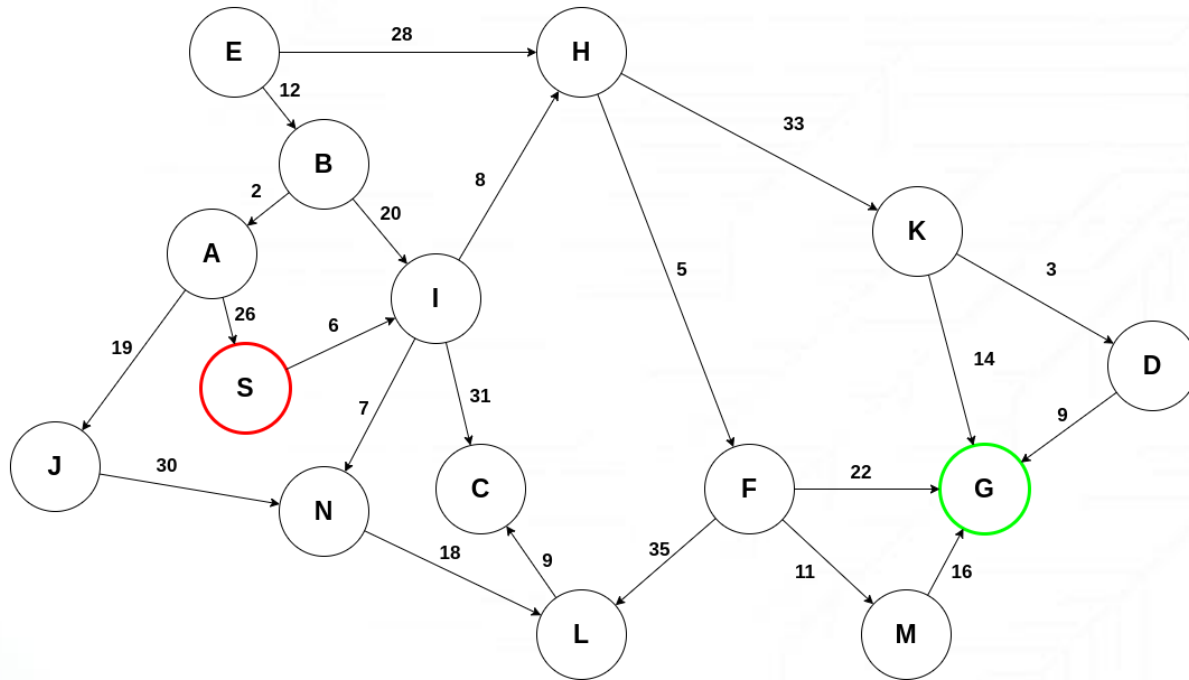


Node[cost]
G[46]

Closed List
S
I
N
H
F
L
M
C

Ejemplo 2 - Explicación

$$\beta = 2$$

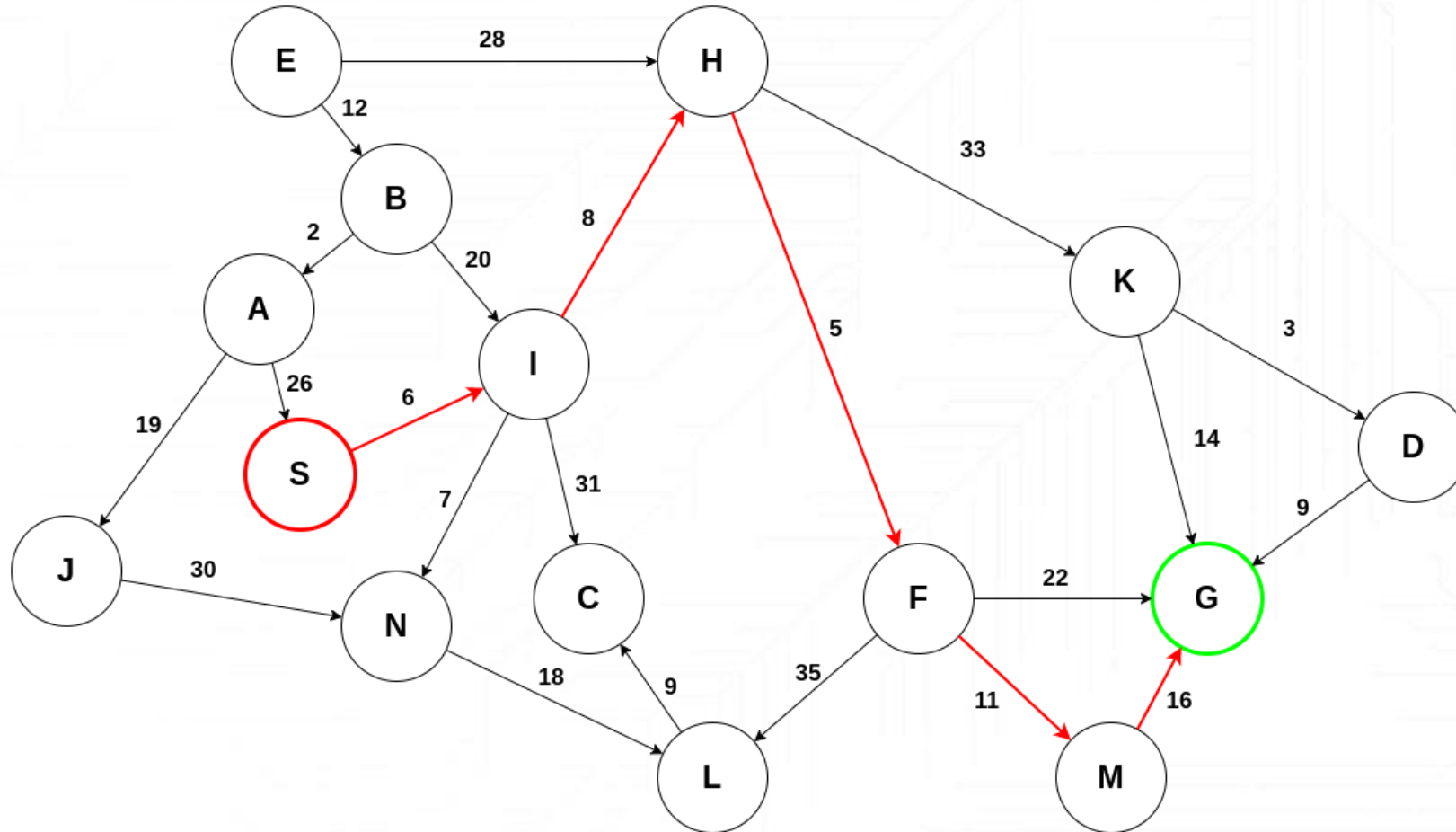


Node[cost]

Closed List
S
I
N
H
F
L
M
C
G

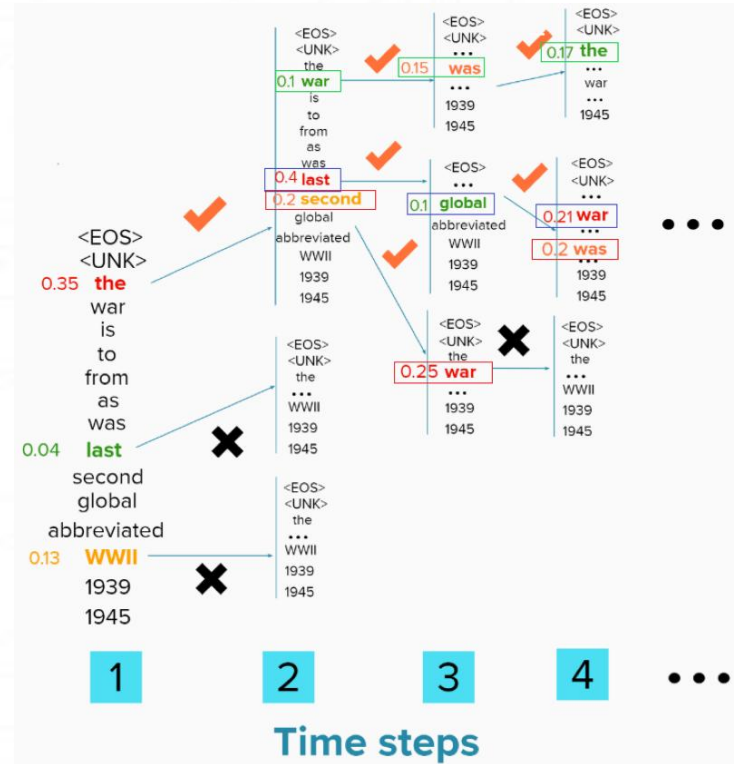
Ejemplo 2 - Explicación

$$\beta = 2$$



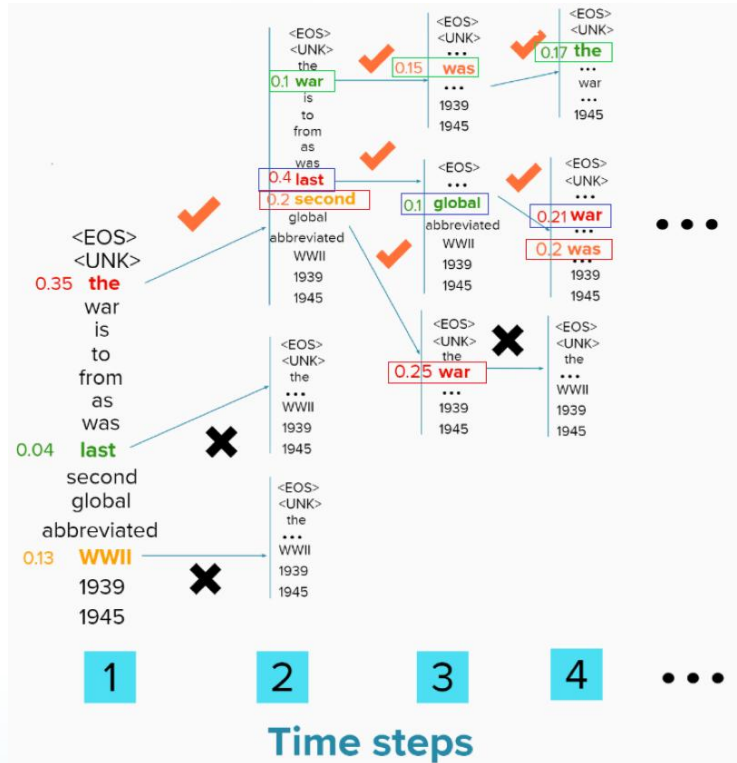
Ejemplo - Procesamiento Lenguaje Natural

Beam_size representa el número de tokens con las mayores probabilidades condicionales en cada paso de tiempo t . En la Figura 2, $beam_size=3$.



https://medium.com/@jessica_lopez/understanding-greedy-search-and-beam-search-98c1e3cd821d

Ejemplo - Procesamiento Lenguaje Natural



Sequence 1 — the last global war — $(0.35 * 0.4 * 0.1 * 0.21) = 0.0029$

Sequence 2 — the second war was — $(0.35 * 0.2 * 0.25 * 0.2) = 0.0034$

Sequence 3 — the war was the — $(0.35 * 0.1 * 0.15 * 0.17) = 0.00089$

Siguiendo el algoritmo de búsqueda voraz, seleccionaríamos la **secuencia 1** , ya que la mayor probabilidad encontrada se encontraba en el segundo token (**last = 0.4**), y la generación de tokens continúa desde esta única rama. Sin embargo, si usamos el algoritmo de búsqueda de haz, la secuencia con mayor probabilidad es la **secuencia 2**. Este es un claro ejemplo de cómo el algoritmo de búsqueda voraz descarta secuencias de tokens con mayor probabilidad



Muchas gracias

